

ExecutionOS User Guide

Version: 1.8.0

Date: 15 September 2026

Status: Living operator guide for the accepted ExecutionOS V2.4 manual-SOD → PRETRADE → Execution Board production core on `main`

Repository: `sibolek/structure-based-trade-management`

Current operating branch: `main`

Last runtime/code checkpoint: `885ee94110e6a91352796c340e6ca40d6d775aff`

Final integrated main tip: `c23faf5c08f3ba60339197be3687fcad37db0e4b`

Frozen downstream execution release: `v2.3.0`

Operating principle: Structure decides. P&L emotion does not.

ExecutionOS preserves pre-entry intent, enforces deterministic risk and authorization boundaries, transfers accepted plans into downstream execution ownership, observes broker reality, and creates an auditable management record. It does not place, replace, cancel, modify, reduce, or flatten broker orders.

Quick Start — Completely New to ExecutionOS? Start Here

This section is deliberately written for someone who has **never used ExecutionOS before**. You do not need to understand the internal architecture before using the normal workflow.

What ExecutionOS does — in plain English

ExecutionOS helps you define a trade **before entry**, checks whether that trade is still valid and appropriately sized, records your authorization, and then watches the broker for the actual fill and trade lifecycle.

Three facts matter most:

1. **ExecutionOS does not place the broker order.** You still place the actual equity order manually in thinkorswim/Schwab.
2. **ARM does not mean “send an order.”** ARM means “I approve this exact trade plan, direction, account and quantity for handoff to the Execution Board.”
3. **The system observes broker reality after ARM.** Once the handoff reaches `LISTENING`, a qualifying Schwab fill can be recognized and promoted to `LIVE`.

If something is `BLOCKED`, stale, inconsistent, or unclear, **do not bypass it by editing files, localStorage, or browser state**. Resolve the stated problem or stop the workflow.

Before your first session

You need:

- this repository on your computer;
- Node/npm dependencies installed;
- working Schwab authorization;

- the intended Schwab account visible to the monitor;
- three Terminal windows or tabs;
- a browser for the ExecutionOS UI.

From the repository folder, make sure you are on the current operating branch:

```
git checkout main
git pull --ff-only origin main
```

If this is the first time you are running the project, or dependencies changed, run:

```
npm install
```

Start ExecutionOS — three Terminal windows

Keep all three processes running during the session.

Terminal 1 — start the read-only Schwab monitor

```
npm run schwab:monitor
```

This process reads broker/account/execution information. It does **not** place orders.

Wait until it reports healthy broker state before relying on ExecutionOS for a live workflow.

Terminal 2 — start the V2.4 PRETRADE service

```
npm run v24:pretrade
```

Default address:

```
http://127.0.0.1:8788
```

This service owns the authoritative PRETRADE workflow: candidate state, trigger/permission evaluation, review, quantity authority and ARM.

Terminal 3 — start the browser UI

```
npm run dev
```

Then open the URL Vite prints, normally:

```
http://localhost:5173
```

What you should verify before using a candidate

Before working a trade candidate, confirm:

- the Schwab monitor is healthy;
- the PRETRADE service is healthy;
- the browser UI is open;

- the intended account shown by ExecutionOS is the account you expect;
- broker positions shown by the system match reality;
- router health is not `BLOCKED`, `ERROR`, or unexpectedly stale.

Useful health checks:

```
curl http://127.0.0.1:8787/health
curl http://127.0.0.1:8787/api/state
curl http://127.0.0.1:8788/health
curl http://127.0.0.1:8788/api/candidates
```

The normal trade workflow — from candidate to LIVE

For a completely new operator, think of the workflow as this sequence:

1. **MANUAL SOD / CANDIDATE IMPORT**
Generate the manual ChatGPT SOD and its individual candidate JSON files.
Each one-candidate bundle has top-level ingressPolicy: MANUAL_AUTHORIZED.
In SOD, use Import Candidate JSON: file / paste / drop → Preview.
Click Import into PRETRADE explicitly; valid new admission is ACCEPTED / WAITING.
2. **WAITING / TRIGGER EVALUATION**
ExecutionOS waits for the candidate's trigger conditions and required evidence.
3. **STRUCTURE / CONTEXT ASSESSMENT**
You provide required operator assessments when the workflow asks for them.
4. **PERMISSION**
ExecutionOS evaluates authoritative trigger, structure, DSS, Phase 4 risk and context evidence.
5. **READY / CAUTION / PASS**
READY = eligible to proceed to review.
CAUTION = may proceed only with the required acknowledgement.
PASS = do not authorize this candidate under the current evidence.
6. **REVIEW**
Inspect the exact account, entry assumptions, stop/risk information and quantity limits.
7. **SELECT QUANTITY**
Use `Final Allowed` as the maximum selectable quantity. You may choose less. Never choose more.
8. **ARM**
Confirm the exact direction and selected quantity. ARM authorizes the plan; it does not place an order.
9. **HANDOFF / LISTENING**
Let the system move the authorization through PENDING → CLAIMED → PREPARED → LISTENING.
10. **PLACE THE BROKER ORDER MANUALLY**
In the normal equity workflow, place the actual order yourself in thinkorswim/Schwab only after the accepted authorization is listening.
11. **LIVE**
A qualifying exact-account Schwab fill can establish LIVE ownership.
12. **EXIT / HISTORY**
Broker fills remain authoritative. When the position becomes flat and exit classification is complete, the trade moves to History.

The quantity numbers — which one do I actually use?

When the review screen shows several quantity fields, **the one that controls your selectable maximum is Final Allowed**.

Field	Plain-English meaning	What you do with it
Phase 4 Stop-Risk Max	Largest quantity that fits the actual expected-entry-to-effective-stop risk budget	Inspect it; it is one input to the final ceiling

Field	Plain-English meaning	What you do with it
2-ATR Volatility Max	Largest quantity that fits the separate 2-ATR volatility-stress safety policy	Inspect it; it can be more restrictive than Phase 4
Reviewed Ceiling	Highest quantity approved for this candidate/version at explicit review	Treat it as non-expanding unless you perform a new explicit review
Final Allowed	The current minimum of the applicable ceilings	This is your maximum selectable quantity. You may choose less; never choose more.

A smaller selected quantity is allowed when it satisfies the instrument's valid quantity rules. **Final Allowed** is a ceiling, not a recommendation to use the maximum.

What does “structural evidence” mean?

If you select:

STRUCTURE = VALID

ExecutionOS requires a short evidence/reference note explaining **what chart structure you are relying on and where you observed it**.

Examples of the level of specificity expected:

2m H2 above VWAP; signal-bar low 581.25; observed 09:42 ET
 5m ORH breakout/retest holding 184.60; chart observed 10:07 ET
 2m micro double bottom at PMH; second low held; chart observed 09:51 ET

These are examples of evidence formatting, not instructions to take those trades. The point is to leave an auditable reference rather than simply writing “looks good.”

Before you click ARM

Check each item:

- ☐ This is the intended candidate/version.
- ☐ Symbol is correct.
- ☐ Direction is correct.
- ☐ Exact execution account is correct.
- ☐ If **STRUCTURE = VALID**, structural evidence/reference is present.
- ☐ Structural invalidation and effective stop are understood and have not been tightened merely to make size fit.
- ☐ You have reviewed Phase 4 Stop-Risk Max, 2-ATR Volatility Max, Reviewed Ceiling and Final Allowed.
- ☐ Selected quantity is **no greater than Final Allowed**.
- ☐ If the candidate is **CAUTION**, the current review package has been explicitly acknowledged.
- ☐ Entry mode shown by the review package is the intended one.
- ☐ You understand that **ARM does not place a broker order**.

After successful ARM, allow the handoff to reach **LISTENING** before following the normal manual broker-entry workflow.

Seven terms you will see constantly

Term	Meaning
Candidate	Proposed trade contract; not yet an authorization or broker position
PRETRADE	The validation/review stage before authorization
READY	Current evidence permits review/authorization to continue
CAUTION	May continue only after explicit acknowledgement of the current package
ARM	Explicit authorization of the exact plan/direction/quantity; not an order
LISTENING	Durable pre-fill state in which the authorized handoff is waiting for qualifying broker evidence
LIVE	ExecutionOS has recognized qualifying broker fill ownership for the authorized trade

For the detailed workflow, continue through this guide. For a normal daily checklist after you are familiar with the system, see **Section 20 — Recommended daily operating procedure**.

Table of contents

- [Quick Start — Completely New to ExecutionOS? Start Here](#)

1. [Purpose](#)
 2. [Current system status](#)
 3. [Architecture and authority](#)
 4. [Risk model](#)
 5. [Installation and startup](#)
 6. [Schwab authorization and account checks](#)
 7. [Candidate and PRETRADE workflow](#)
 8. [Phase 3 DSS, Phase 4 risk sizing, and PRETRADE quantity safety](#)
 9. [Operator review, CAUTION and ARM](#)
 10. [Handoff and pre-fill execution ownership](#)
 11. [LIVE management](#)
 12. [Authorization Exceptions and reconciliation](#)
 13. [Persistence and recovery](#)
 14. [End-of-Day reporting](#)
 15. [Futures / NinjaTrader status](#)
 16. [Security](#)
 17. [Troubleshooting](#)
 18. [Current limitations and deferred work](#)
 19. [Repository discipline](#)
 20. [Recommended daily operating procedure](#)
 21. [Command reference](#)
 22. [Documentation map and glossary](#)
-

1. Purpose

This is the practical operator guide for ExecutionOS after the September 15, 2026 production-core manual-SOD fast-forward merge to `main`. It includes Decision 28 admission integrity, Manual Candidate Import, and individual manual-SOD deliverables, followed by the accepted PRETRADE → manual ARM → Execution Board workflow and September 8 quantity-safety controls.

Use it to answer:

How do I operate the accepted system today, what authority owns each decision, what may I do from the browser, and what remains intentionally unavailable?

For frozen architecture and approved policy use:

```
docs/ExecutionOS_V2.4_Design_Baseline_v0.5_APPROVED.md
docs/ExecutionOS_V2.4_Design_Baseline_v0.5_Traceability_Audit_APPROVED.md
docs/ExecutionOS_V2.4_PRETRADE_Quantity_Safety_Addendum_v0.1_APPROVED.md
```

For implementation acceptance and final merge closeout use:

```
docs/ExecutionOS_V2.4_Execution_Board_Handoff_Integration_Closeout_2026-09-06.md
docs/ExecutionOS_V2.4_Execution_Board_Handoff_Final_Merge_Closeout_2026-09-08.md
docs/ExecutionOS_V2.4_Production_Core_Manual_SOD_Final_Merge_Closeout_2026-09-15.md
```

Frozen approved historical documents remain approval-time evidence and should not be rewritten merely because implementation status advanced.

2. Current system status

2.1 Frozen downstream reference

```
v2.3.0
baabb75f36050599f20e6c89e8db2f1f7d7769a1
```

Legacy/manual V2.3 remains the reference for its own historical workflow and trusted downstream execution concepts.

V2.4-origin trades do **not** use the legacy symbol-only / `detectedAt` ownership path. They use exact-account ownership, authoritative Schwab `executionTime`, and the lossless ownership journal.

2.2 Accepted V2.4 state on `main`

The accepted implementation now on `main` includes:

1. Candidate Ingestion;
2. MarketDataProvider;

3. DSS / Micro-Volatility Buffer;
4. Effective-Stop Risk Sizing;
5. PRETRADE → ARM → Execution Board handoff integration;
6. TODO #18 structural-evidence UX enforcement;
7. TODO #19 PRETRADE quantity-safety policy;
8. accepted manual SOD / trade-card ingestion carried forward from its September 11 checkpoint;
9. Decision 28 admission-anchored candidate integrity;
10. Manual Candidate Import with local Preview and explicit canonical PRETRADE admission;
11. Manual SOD individual candidate deliverables with top-level `ingressPolicy: "MANUAL_AUTHORIZED"`.

Last runtime/code checkpoint:

```
885ee94110e6a91352796c340e6ca40d6d775aff
```

Final integrated `main` tip after the production-core status documentation:

```
c23faf5c08f3ba60339197be3687fcad37db0e4b
```

The September 15 fast-forward includes the accepted predecessor history plus Decision 28 integrity, Manual Candidate Import, and Manual SOD individual deliverables. Later documentation-only commits may advance `main` without changing the runtime/code checkpoint. The September 8 handoff checkpoint `26ad8f86d2f0b4af96c186b26f250f4bb10a9dec` remains historical acceptance evidence.

The former feature branch `v24-execution-board-handoff` was retired and deleted after the verified fast-forward merge. It is not an operating branch.

Slices 1–7 are accepted. Decisions 22–97 remain frozen. GitHub issues #18 and #19 are completed/closed.

Governing invariant:

V2.4 authorizes; the handoff transfers; V2.3-compatible execution infrastructure owns execution.

2.3 Final accepted lifecycle

```
MANUAL CHATGPT SOD
→ INDIVIDUAL ONE-CANDIDATE JSON BUNDLE
  (top-level ingressPolicy: MANUAL_AUTHORIZED)
→ MANUAL CANDIDATE IMPORT / PREVIEW
→ EXPLICIT IMPORT
→ CANONICAL PRETRADE: ACCEPTED / WAITING
↓
PRETRADE_TRIGGER_EVALUATING
↓
PERMISSION_EVALUATING
↓
READY / CAUTION / PASS
↓
OPERATOR REVIEW
↓
ARM
↓
IMMUTABLE EXECUTION BOARD HANDOFF
↓
PENDING
↓
CLAIMED
↓
PREPARED
↓
LISTENING
↓
EXACT-ACCOUNT SCHWAB OPENING FILL
↓
LIVE
↓
ENTRY_FRAGMENT / ADD / PARTIAL / FLAT / REVERSAL
↓
EXIT
↓
OPERATOR EXIT CLASSIFICATION
↓
HISTORY
↓
SYMBOL OWNERSHIP RELEASE
```

2.4 Broker boundary

```
readOnly === true
brokerWriteAuthority === false
```

No V2.4 state grants broker-write authority.

Actual equity order entry remains manual in thinkorswim/Schwab.

2.5 V3 status

V3 Management Governor has not started. It requires a separate explicit design/implementation authorization.

3. Architecture and authority

ExecutionOS deliberately separates operator intent from machine authority.

3.1 Browser role

The browser is:

- presentation;
- operator intent capture;
- read-only inspection;
- serialized management command submission.

The browser is **not** a generic lifecycle authority and cannot directly manufacture READY, CAUTION, PASS, ARMED, LIVE, or ownership state.

3.2 Server-side PRETRADE authority

Server-side PRETRADE services own:

- canonical candidate lifecycle;
- trigger relevance/satisfaction evidence;
- structural validity;
- DSS and permission orchestration;
- review package state;
- quantity-safety evaluation;
- quantity selection validation;
- CAUTION acknowledgement;
- OCO authority;
- final ARM authorization;
- immutable handoff creation and delivery registration.

3.3 Downstream authority

The canonical Execution Board store owns:

- PREPARED/LISTENING installation state;
- retirement state;
- first-fill ownership transfer;
- LIVE lifecycle;
- Slice 7 management state;
- Authorization Exceptions;

- History ownership release.

Browser mutations are serialized through approved canonical writer boundaries and Web Locks.

3.4 Runtime router

The router is default-on and serializes safe downstream work.

Emergency negative switch:

```
VITE_EXECUTIONOS_V24_ROUTER_DISABLED=true
```

Semantics:

```
unset / false -> enabled  
true         -> PAUSED  
other nonempty value -> BLOCKED / fail closed
```

Health states:

```
RUNNING  
WAITING_FOR_SCHWAB  
WAITING_FOR_PRETRADE  
WAITING_FOR_ROUTER_LOCK  
PAUSED  
STALE  
BLOCKED  
ERROR
```

Reconciliation is trade/ownership state, not router health.

4. Risk model

Permanent project rule:

Maximum planned price risk per trade = 0.5% of the exact relevant trading-account equity.

Required hierarchy:

```
STRUCTURE
↓
STRUCTURAL INVALIDATION
↓
PHASE 3 EFFECTIVE STOP
↓
CURRENT EXPECTED ENTRY
↓
0.5% RISK BUDGET
↓
PHASE 4 STOP-RISK MAXIMUM
↓
PRETRADE QUANTITY-SAFETY CEILING
↓
EXPLICIT OPERATOR QUANTITY
```

Never choose size first and tighten the stop until the dollars fit.

4.1 Structural invalidation vs effective stop

They are separate concepts.

Structural invalidation answers:

Where is the trade thesis wrong?

Effective stop answers:

What volatility-protected execution stop does Phase 3 derive from that invalidation?

Do not substitute one for the other.

4.2 Phase 3 stop policy

Accepted V1 DSS policy includes:

- 2-minute Wilder ATR(14);
- RTH-only reconstruction;
- $ATR \times 0.30$ volatility buffer;
- directionally protective price-increment rounding.

Phase 3 owns the calculated pre-entry effective stop.

4.3 Phase 4 affordability

Phase 4 sizes against the effective stop.

It may:

- return a valid maximum quantity;
- return `NO_AFFORDABLE_SIZE`;

- block on missing/stale evidence;
- fail closed on invalid metadata.

It may never alter the effective stop to make size fit.

Phase 4 `riskDistance` and `plannedDollarRisk` continue to represent actual expected-entry-to-effective-stop economics.

4.4 Separate PRETRADE quantity safety

The accepted quantity-safety policy is a **separate ceiling** layered on top of Phase 4. It is not a replacement for Phase 4 sizing and it does not create a synthetic stop.

It uses authoritative DSS 2-minute Wilder ATR(14):

```
volatilityStressDistance = 2.0 × ATR
```

For equities:

```
volatilityRiskPerUnit = 2.0 × ATR
```

For futures, the 2-ATR distance is converted to ticks with protective upward rounding and multiplied by trusted tick value.

The volatility maximum is derived from the same existing max-dollar-risk budget and floored to the instrument's valid quantity increment.

Current policy maximum:

```
policyMaxQuantity = min(
  phase4MaxAffordableQuantity,
  volatilityMaxQuantity
)
```

This guard prevents a marketable expected entry very near the effective stop from mechanically expanding permitted quantity solely because the Phase 4 per-unit stop risk became very small.

The quantity-safety policy does **not**:

- move structural invalidation;
- move or synthesize the effective stop;
- rewrite Phase 4 `riskDistance`;
- rewrite Phase 4 `plannedDollarRisk`;
- create a buying-power, notional, or margin rule.

4.5 Reviewed quantity ceiling

At the first explicit operator review, the current policy maximum is frozen as the candidate/version **reviewed quantity ceiling**.

At ARM-time fresh revalidation:

```
finalAllowedQuantity = min(  
    freshPhase4Max,  
    freshVolatilityMax,  
    priorReviewedCeiling  
)
```

Therefore fresh ARM-time evidence may reduce an already-reviewed ceiling but may not increase it without a new explicit operator review.

4.6 Live risk after fills

After LIVE begins, actual fill economics become relevant without rewriting expected-entry provenance.

The authorization budget remains finite.

For exposure increases:

```
resulting quantity <= applicable live ceiling
```

and:

```
cumulative realized losses attributable to authorization  
+ worst-case remaining open loss after increase  
<= original authorizedMaxDollarRisk
```

Realized losses consume capacity. Realized profits do not replenish it.

5. Installation and startup

5.1 Branch rule

Operate the accepted V2.4 system from `main`:

```
git checkout main  
git pull --ff-only origin main
```

Do not use the retired `v24-execution-board-handoff` branch for normal operation.

5.2 Install

```
npm install
```

5.3 Normal equity session — three terminals

Terminal 1 — Schwab monitor

```
npm run schwab:monitor
```

Wait for healthy read-only broker state.

Terminal 2 — V2.4 PRETRADE/handoff service

```
npm run v24:pretrade
```

Default service:

```
http://127.0.0.1:8788
```

Terminal 3 — UI

```
npm run dev
```

Typical URL:

```
http://localhost:5173
```

5.4 Startup checks

Confirm:

- current Git branch is `main`;
- Schwab monitor is healthy;
- exact intended execution account is present;
- PRETRADE service is healthy;
- router health is appropriate;
- another tab is not unexpectedly holding leadership;
- broker positions match reality.

Health endpoints:

```
curl http://127.0.0.1:8787/health
curl http://127.0.0.1:8787/api/state
curl http://127.0.0.1:8788/health
curl http://127.0.0.1:8788/api/candidates
```

6. Schwab authorization and account checks

6.1 Environment

Local environment example:

```
SCHWAB_CLIENT_ID=...  
SCHWAB_CLIENT_SECRET=...  
SCHWAB_CALLBACK_URL=https://127.0.0.1:8182
```

Never commit `.env.local`, secrets, or token files.

6.2 Authorize

```
npm run schwab:auth
```

6.3 Account check

```
npm run schwab:account
```

Phase 4 account-equity authority is:

```
currentBalances.liquidationValue
```

Phase 4 does not silently substitute cash, buying power, initial balances, another account, or `currentBalances.equity` when liquidation value is absent.

7. Candidate and PRETRADE workflow

A candidate is a pre-entry contract proposal, not a broker position.

A complete proposal should include:

- symbol;
- direction;
- setup;
- primary timeframe;
- thesis;
- trigger;
- structural invalidation;
- targets;
- management contract/intent;
- finite validity window.

7.1 Manual ChatGPT SOD and Candidate Import

Manual ChatGPT SOD is the current production-generation workflow. Supply charts/screenshots to ChatGPT/ manual analysis and request `manual-sod-analysis-YYYY-MM-DD.json` as a **standard deliverable**, containing the report's structured `artifactContent`, authored `candidateProposals`, and `bundleMetadata`. Use the [transport template](#) for the outer shape; candidate contracts still come from the current repository's existing candidate template and validator.

Serialize already-authored analysis and package it in the standard Downloads location (if the transport already exists, run only the package command with its actual path):

```
npm run v24:manual-sod-analysis -- authored-sod.json 2026-09-16
# Verify the transport exists, is readable, and matches the authored analysis.
npm run v24:manual-sod-package -- ~/Downloads/ExecutionOS/SOD/2026-09-16-transport/manual-sod-analysis-2026-09-16.json
```

Choose the actual session date and a fresh output directory. The morning flow is **charts/screenshots → ChatGPT/manual analysis → dated analysis JSON → packager → reports/dashboard always for valid reporting input; candidate JSON only when the current local validator passes**. Check `candidate-delivery-status.json` for `DELIVERED`, `WITHHELD` and its reason, or `NO_CANDIDATES`.

There is no local upstream generator for manual chart analysis. If authored data needs the standard filename, use

```
npm run v24:manual-sod-analysis -- <authored-sod.json> <YYYY-MM-DD> [output-directory]. This optional serializer creates transport only; no screenshots, AI request, service startup, candidate validation, import, or state change is involved. It does not automatically retrieve analysis from ChatGPT.
```

PRETRADE, Schwab, and Execution Board may remain offline for both steps. Reporting uses the existing 19-section renderer and dashboard styling. An unavailable validator or failed candidate validation withholds the entire candidate package while reports still complete; malformed report content or report filesystem errors still fail. Each delivered file in `individual-candidates/` is a canonical bundle containing exactly one unchanged candidate and **top-level** `"ingressPolicy": "MANUAL_AUTHORIZED"`. This policy belongs on the bundle, never inside the candidate or its provenance. Import only these validated individual files; the intermediate analysis JSON is not an import file.

The candidate's identity, version, source, provenance, trade logic, and optional content must be preserved. Do not inject a missing policy into a supplied bundle or relabel automated output to make it importable. Correct manual-package authoring at the source. The combined A+ archival/export bundle keeps its existing shape and policy behavior; use the individual manual import files for this workflow.

See the [Manual SOD Deliverable Specification](#) and its [individual candidate template](#). Template placeholders must be completed during authoring before a candidate is finalized.

Standard SOD output layout (use the actual session date):

```
~/Downloads/ExecutionOS/SOD/
├── YYYY-MM-DD/
│   ├── report.md
│   ├── report.html
│   ├── dashboard.html
│   ├── candidate-delivery-status.json
│   ├── individual-candidates/          # only when DELIVERED
├── YYYY-MM-DD-transport/
│   └── manual-sod-analysis-YYYY-MM-DD.json
```

Ad-hoc standalone trade card from a chart

Ask ChatGPT, for example, **“create a trade card for this chart.”** Normal authoring delivers the standalone HTML and `manual-trade-card-analysis-YYYY-MM-DD-SYMBOL.json`. The transport carries the exact authored HTML, one structured candidate proposal from the same analysis, and available manual bundle metadata. It is not an import file. Use the [transport-shape template](#); the current local validator remains the candidate contract authority.

```
npm run v24:manual-trade-card-analysis -- authored-trade-card.json 2026-09-16 NVDA
# Verify the transport exists, is readable, and matches the authored analysis.
npm run v24:manual-trade-card-package -- ~/Downloads/ExecutionOS/TradeCards/2026-09-16/NVDA-transport/manual-trade-card-analysis-2026-09-16-NVDA.json
```

Choose a fresh output directory. The command writes `trade-card.html` without changing the author's styling, plus `candidate-delivery-status.json`. Only `DELIVERED` includes an individual candidate JSON ready for Preview. `WITHHELD` explains why the optional candidate could not be delivered; the authored HTML remains available. A `PENDING` status is incomplete and must not be imported. The ordinary flow is **chart screenshot → ChatGPT/manual analysis → transport → local package → HTML; validated candidate if available → Preview → explicit Import**.

No full SOD report or running PRETRADE/Schwab/Execution Board/broker service is required. Chart analysis remains upstream; the local command neither analyzes screenshots nor calls a model. Candidate output reuses the manual SOD individual-file gate and one-candidate `MANUAL_AUTHORIZED` bundle. Import behavior and all review/ARM/execution boundaries below are unchanged.

For local serialization, use

```
npm run v24:manual-trade-card-analysis -- <authored-trade-card.json> <YYYY-MM-DD> <SYMBOL> [fresh-transport-directory]
```

Write the transport as a standalone top-level artifact in a fresh writable location, outside existing SOD output directories. The caller must verify existence, readability, and content before reporting success. On permission/write failure, retry only the preserved serialized bytes at a fresh path, without regenerating analysis. Existing transport files are never overwritten; neither CLI retries automatically. See the [complete standalone procedure and statuses](#).

Standard standalone trade-card output layout:

```
~/Downloads/ExecutionOS/TradeCards/YYYY-MM-DD/
├── SYMBOL/
│   ├── trade-card.html
│   ├── candidate-delivery-status.json
│   └── 01-<safe-candidateId>.json      # only when DELIVERED
├── SYMBOL-transport/
│   └── manual-trade-card-analysis-YYYY-MM-DD-SYMBOL.json
```

Manual output path rules

The final output-directory argument is optional on both manual serializers and both manual packagers. Package defaults come only from the standard transport **filename** (valid calendar date and, for trade cards, a safe uppercase symbol label), never the clock or candidate fields. Labels do not rewrite or validate authored dates/symbols; authors must keep them consistent. An undated or renamed input still works with an explicit output directory. The `-TRANSPORT` suffix (case-insensitive) is reserved for transport-directory naming only in default trade-card path derivation; labels such as `NVDA-TRANSPORT` require explicit output directories for transport and package destinations.

Transport defaults use a fresh sibling directory so the transport never occupies an existing or future package directory. Both default transport directories and package directories must be absent, including

empty directories and symlinks. A repeated run fails; there is no overwrite, auto-suffix, automatic retry, or silent fallback. For a new version or write-only recovery, supply an explicit fresh output directory as the last argument; preserve and verify the existing serialized analysis. Explicit transport destinations retain their existing exclusive-file behavior. Never choose an existing package as a transport destination.

CLI input and output paths expand `~` or `~/` using the current user's home directory, including when quoted; `~otheruser` is rejected. Default mode requires an existing writable `~/Downloads` directory. Missing/unavailable Downloads fails clearly and allows an explicit destination override. Other filesystem errors name the failing path; partial or interrupted output must be retried at a new path. Programmatic writer APIs still require explicit output paths. No changes to automated Production SOD, analysis, validator gating, Manual Import, PRETRADE, permission checks, ARM, execution, or broker authority.

Load and Preview

In the **SOD** workspace, find **Import Candidate JSON** (the Manual Candidate Import control).

1. Choose one `.json` file with **Choose JSON file**, drop one `.json` file into the import area, or use **Paste candidate JSON**. The input limit is 768 KiB and exactly one candidate at a time.
2. File selection and drop automatically open the local preview. After pasting, click **Preview JSON**. Editing the input clears the previous preview/result and requires another Preview.
3. Inspect symbol, direction, setup, trigger, structural invalidation, targets, source/date, candidateId, and contractVersion against the intended trade. Optional candidate content is preserved even when it is not shown in the summary.

Preview parses and checks the input locally. **It sends no import request, creates no PRETRADE candidate, and does not prove server admission, permission, or risk approval.** Preview is available while PRETRADE is offline; Import is disabled until the service is connected. Canonical validation, identity, validity, conflict, and supersession checks occur on the server when you import.

The control also accepts supported standalone canonical candidates and existing standalone trade-card envelopes. Existing canonical bundles must already carry `MANUAL_AUTHORIZED` ; they are not silently converted from another ingress policy.

Explicit Import and results

Click **Import into PRETRADE** only after reviewing the preview. Read the **Canonical import result**, then use **Open PRETRADE** to inspect current state.

Result	Meaning and next step
ACCEPTED / WAITING	A valid new candidate was admitted as a proposal. Continue the existing PRETRADE trigger, permission, risk, review, quantity-selection, and manual ARM workflow.
DUPLICATE / Already imported	The unchanged candidate/version is already admitted. Use that existing candidate; a retry does not create a second candidate or reset its lifecycle to WAITING.
CONFLICT	The same version has different content. Resolve the contract/version conflict; do not force replacement.
STALE	An older version was not imported. Inspect the current candidate.
ACTION_REQUIRED	The prior candidate is unchanged. Complete the existing manual supersession review/ authorization process before importing a replacement.
REJECTED	Read the validation reasons and correct the proposal at its source.

WAITING at admission records the admission result. PRETRADE shows the current lifecycle, including any validity expiration or later operator action. If delivery is uncertain, check PRETRADE and retry the same unchanged contract to resolve the outcome; do not invent a new identity to bypass uncertainty. An integrity error stops import and must be resolved through the existing recovery process, not by editing stored authority.

Import grants no activation, permission, risk approval, ARM authorization, Execution Board trade/handoff, execution ownership, or broker-write authority. MANUAL_AUTHORIZED selects the manual admission path; it does not authorize a trade. Existing permission/risk checks, explicit review, quantity selection, and manual ARM remain mandatory. Actual equity order entry remains manual in thinkorswim/Schwab after the authorized handoff reaches LISTENING.

Canonical admission and Decision 28 integrity

Decision 28 anchors the admitted candidate's substantive JSON, including arbitrary optional content, to its accepted immutable contract and admission record. Runtime bookkeeping cannot silently redefine the admitted contract. Tampering or missing/contradictory integrity evidence fails closed; import cannot supply system-owned lifecycle or authorization fields.

Canonical bundles enter through:

```
POST /api/candidates/import
```

Source adapters may not bypass canonical validation, versioning, validity, WAITING state, or authorization rules.

7.2 WAITING means proposal only

An imported WAITING candidate:

- does not own broker fills;
- is not ARMED;
- is not automatically advanced by the downstream router.

The operator/server PRETRADE workflow must advance it.

7.3 Active projection

The PRETRADE workspace projects active unarmed states including:

```
WAITING
PRETRADE_TRIGGER_EVALUATING
PERMISSION_EVALUATING
READY
CAUTION
```

Terminal unarmed states include:

PASS
EXPIRED
INVALIDATED
DECLINED
SUPERSEDED
OCO_CANCELLED

Unknown states remain visible and fail closed rather than being guessed into a normal status.

7.4 Trigger evaluation

Trigger relevance is separate from satisfaction.

A trigger may use:

- deterministic quote/bar evidence;
- structured compound nodes;
- explicit operator/manual confirmation where allowed.

Satisfaction provenance is durable and must identify the exact trigger branch/evidence/version.

7.5 Structural evidence requirement

When the operator selects:

STRUCTURE = VALID

a non-empty structural evidence/reference is required before permission submission.

The UI:

- labels the field as required for `VALID` ;
- displays an inline validation message when missing;
- disables `EVALUATE PERMISSION` while required evidence is blank.

Backend enforcement remains authoritative:

MISSING_STRUCTURE_PROVENANCE

The browser validation does not weaken or replace the backend contract.

A useful evidence/reference should identify **what structure was observed and where/when it was observed**. Examples:

2m H2 above VWAP; signal-bar low 581.25; observed 09:42 ET
5m ORH breakout/retest holding 184.60; observed 10:07 ET
2m micro double bottom at PMH; second low held; observed 09:51 ET

These examples demonstrate evidence formatting only. They do not make any setup automatically valid and are not instructions to take a trade.

7.6 Permission evaluation

Permission combines authoritative evidence for:

- trigger satisfaction;
- structural validity and provenance;
- Phase 3 DSS;
- Phase 4 risk sizing;
- PRETRADE quantity-safety inputs where review is applicable;
- account/entry evidence;
- macro/setup context from trusted evaluator or explicit operator assessment.

Possible outcomes:

```
READY
CAUTION
PASS
```

Unresolved data may remain retryable or integrity-blocked rather than being mislabeled PASS.

8. Phase 3 DSS, Phase 4 risk sizing, and PRETRADE quantity safety

8.1 Expected entry

MARKETABLE_NOW

```
LONG  = ask
SHORT = bid
```

STOP_TRIGGER

```
LONG  = max(triggerPrice, ask)
SHORT = min(triggerPrice, bid)
```

No fallback to last, mark, or candle close is allowed.

Quote requirements include:

- positive bid/ask;
- `bid <= ask`;
- locked market allowed;
- crossed market blocked;
- quote age ≤ 5 seconds.

8.2 Risk geometry

```
LONG riskDistance = currentExpectedEntry - effectiveStop
SHORT riskDistance = effectiveStop - currentExpectedEntry
```

Invalid direction-aware geometry blocks.

8.3 Risk budget

```
rawMaxDollarRisk = accountEquity × 0.005
maxDollarRisk = floorToCent(rawMaxDollarRisk)
```

Budget rounding never rounds upward.

8.4 Equity Phase 4 sizing

```
riskPerShare = riskDistance
rawQuantity = maxDollarRisk / riskPerShare
phase4MaxAffordableQuantity = floor(rawQuantity)
```

Odd lots are allowed. Fractional shares are not assumed.

8.5 Futures Phase 4 sizing

Trusted futures metadata includes:

```
tickSize
tickValue
pointValue? // optional consistency check
minimumQuantity
quantityIncrement
currency
```

Calculation:

```
riskTicks = ceil(riskDistance / tickSize)
riskPerContract = riskTicks × tickValue
```

8.6 Phase 4 affordability outcomes

```
VALID
NO_AFFORDABLE_SIZE
BLOCKED
ERROR
```

`NO_AFFORDABLE_SIZE` maps downstream to:

Do not tighten the stop to force affordability.

8.7 PRETRADE quantity-safety evaluation

The review layer evaluates the separate 2-ATR volatility maximum from authoritative DSS ATR evidence.

The review UI exposes:

- **Phase 4 Stop-Risk Max;**
- **2-ATR Volatility Max;**
- **Reviewed Ceiling;**
- **Final Allowed;**
- the binding constraint;
- ATR / stress-distance audit context.

For day-to-day operation, use this interpretation:

Field	Meaning	Operator action
Phase 4 Stop-Risk Max	Largest quantity fitting actual expected-entry-to-effective-stop risk	Inspect; this is one ceiling
2-ATR Volatility Max	Largest quantity fitting the separate volatility-stress guard	Inspect; this may become the binding ceiling
Reviewed Ceiling	Maximum quantity previously accepted for this candidate/version at explicit review	Treat as non-expanding without a new explicit review
Final Allowed	Current minimum of all applicable ceilings	Use this as the maximum selectable quantity; choose less if desired, never more

The UI also makes explicit that the effective stop is unchanged.

If the policy inputs are invalid or no valid safe quantity exists, the workflow fails closed rather than manufacturing a quantity.

8.8 ARM freshness

Every final ARM attempt requires fresh Phase 4 and applicable quantity-safety evidence.

At authorization:

- quote freshness ≤ 5 seconds;
- account snapshot freshness ≤ 15 seconds.

Fresh ARM-time calculations may lower **Final Allowed**; they may not expand beyond the prior reviewed ceiling without a new explicit review.

9. Operator review, CAUTION and ARM

9.1 Review package

Review is bound to a material `reviewPackageId`.

The package exposes authorization-critical facts including:

- expected entry;
- structural invalidation;
- effective stop;
- max dollar risk;
- Phase 4 Stop-Risk Max;
- 2-ATR Volatility Max;
- Reviewed Ceiling;
- Final Allowed quantity;
- binding quantity constraint;
- exact execution account;
- permission outcome/provenance.

Material changes produce a new review package and clear stale quantity/acknowledgement state as required by the authoritative review contract.

9.2 Quantity selection

`Final Allowed` is a **ceiling**, not a required size.

Operator rule: `Final Allowed` is the maximum quantity you may select from the current review. You may always choose less when valid for the instrument. Never select more.

Select a valid explicit quantity within instrument increment/minimum rules and no greater than the current final allowed quantity.

The final ARM action must confirm the exact selected quantity.

9.3 Non-expanding reviewed ceiling

The first explicit review freezes the candidate/version reviewed quantity ceiling.

If fresh quote/ATR/account evidence later reduces safe quantity, the ceiling may ratchet downward.

It may not ratchet upward merely because the expected entry moved closer to the stop or current volatility inputs would otherwise allow more size. A higher ceiling requires a new explicit operator review.

9.4 CAUTION acknowledgement

A CAUTION candidate may be authorized only after explicit acknowledgement bound to the exact current review package.

A materially changed package requires a new acknowledgement.

9.5 Before you ARM — operator checklist

Before using the ARM control, verify:

- ☐ intended candidate/version;
- ☐ correct symbol;
- ☐ correct direction;
- ☐ correct exact execution account;
- ☐ structural evidence/reference present when `STRUCTURE = VALID`;
- ☐ structural invalidation understood;
- ☐ effective stop understood and not tightened merely to make quantity fit;
- ☐ Phase 4 Stop-Risk Max reviewed;
- ☐ 2-ATR Volatility Max reviewed;
- ☐ Reviewed Ceiling reviewed;
- ☐ `Final Allowed` reviewed;
- ☐ selected quantity is no greater than `Final Allowed`;
- ☐ current CAUTION package acknowledged when required;
- ☐ entry mode is the intended one;
- ☐ you understand that ARM does **not** place a broker order.

For the normal equity workflow, ARM first, allow the handoff to reach `LISTENING`, and then place the actual broker order manually in thinkorswim/Schwab.

9.6 Final ARM confirmation

ARM itself is the final explicit **quantity/direction** confirmation, for example:

```
ARM NVDA LONG - 25 SHARES
```

The exact account is already exposed in the authoritative current review package and is carried/frozen by the ARM request. The request also binds the exact candidate/version, `reviewPackageId`, entry mode, and current structural/context assessments as applicable.

There is no separate final account-confirmation control. If the account or another material authorization fact changes, final revalidation changes the review package or rejects the stale ARM rather than silently accepting the old review.

The server performs fresh permission/risk/quantity-safety revalidation before authorization. Accidental Enter-key submission is not wired to ARM.

9.7 Successful ARM

Successful ARM freezes:

- candidate/version/hash;
- DSS evaluation identity;
- risk evaluation identity;
- account;
- direction;
- selected quantity;

- reviewed/final quantity authority carried by the accepted review;
- authorized time;
- management/entry authorization contract;
- handoff identity.

It then creates/registers exactly one immutable Execution Board handoff and one PENDING delivery.

ARM does **not** place a broker order.

9.8 OCO

OCO groups bind exact candidate versions on the same symbol and common account.

Only one winner may complete ARM. OCO authority does not become broker OCO order-placement authority.

10. Handoff and pre-fill execution ownership

10.1 PENDING → CLAIMED

The receiver claims one delivery using stable receiver identity. A competing receiver may not steal the claim.

10.2 PREPARED

`PREPARED` is a durable local pre-fill reservation while broker proof catches up.

A proposed listening boundary during `PREPARED` is transient and not yet ownership authority.

10.3 LISTENING

`LISTENING` begins only after final admission proof.

It freezes immutable:

```
executionListeningAt
```

After `LISTENING` becomes durable, transport loss does not release ownership.

10.4 First-entry authorization deadline

ARM also freezes a finite first-entry authorization deadline.

Rules:

1. A qualifying opening fill must have authoritative `executionTime` before the deadline.
2. The deadline is never extended because of transport delay, browser restart, or recovery.
3. If no opening fill occurs before the boundary, the authorization becomes unfilled/retired and fresh interest requires a new `PRETRADE` + `ARM` cycle.

4. Recovery may prove a timely fill only from authoritative broker evidence.
5. A fill exactly at the cutoff is late when the contract requires `executionTime < cutoff`.

10.5 DISCARD before fill

PREPARED may be discarded before listening ownership begins.

LISTENING discard freezes one immutable retirement cutoff.

Possible outcomes include:

```
REQUESTED
RETIRED
SUPERSEDED_BY_PRIOR_FILL
RECONCILIATION_REQUIRED
```

Healthy broker evidence that is simply behind the cutoff remains REQUESTED while it catches up.

10.6 First-fill ownership

Ownership requires:

- exact authorized account;
- matching symbol;
- matching opening direction/effect;
- authoritative broker `executionTime`;
- continuous execution coverage;
- valid lossless ownership journal;
- required order provenance.

A qualifying partial first fill establishes LIVE immediately.

Promotion atomically creates the durable V2.4 lifecycle and visible V2.4 LIVE trade.

11. LIVE management

Slice 7 adds explicit management authority without creating broker writes.

11.1 Broker truth first

Actual fills are always preserved as broker truth, even when they violate authorization.

ExecutionOS may:

- flag;
- block further adds;
- require reconciliation;
- recompute actual risk.

It may not rewrite a fill away or pretend the broker did something else.

11.2 ARM ceiling

`selectedQuantity` is the immutable maximum simultaneous authorized quantity under the ARM.

It is not a one-shot required entry size.

The PRETRADE reviewed/final quantity safety controls how large `selectedQuantity` may be authorized to become; after ARM, the immutable selected quantity is the downstream ARM ceiling.

11.3 Position-build window

After first fill, the management contract may allow additional build capacity for a finite period.

Unused capacity expires according to the contract.

11.4 Complete Position Build

Complete Position Build explicitly relinquishes any never-used initial build capacity.

After completion:

- ARM ceiling remains immutable audit provenance;
- live ceiling becomes the maximum legitimately established quantity before completion;
- live ceiling may not increase under the same authorization.

11.5 Re-add

A re-add is allowed only when all applicable conditions remain true, including:

- same authorization;
- same symbol/direction/account/trade;
- management contract permits it;
- structure remains valid enough under the frozen rules;
- resulting exposure fits the current live ceiling;
- fresh exposure-increase risk check passes.

11.6 Lifecycle loss budget

The authorization's max-dollar-risk is a finite lifecycle budget.

Realized losses consume capacity. Profits do not restore consumed capacity.

P/L attribution must be supported by lossless broker journal/execution evidence. Ambiguity blocks exposure increases.

11.7 Effective-stop authority

The live effective stop may change only through:

- explicit operator management action; or
- a deterministic frozen rule authorized by the contract.

Changes preserve old/new value, time, source and reason.

A tighter valid stop may free risk within existing quantity ceilings.

A wider stop cannot manufacture new quantity/risk authority.

Do not tighten the effective stop merely to make the risk number fit.

Changing the OS stop does **not** modify the broker stop.

11.8 Targets

Targets are structured/versioned management state.

Recording target attainment:

- updates OS management state;
- does not submit a broker exit;
- does not create broker-write authority.

11.9 Discretionary notes

Discretionary notes are audit context only unless a specific structured command says otherwise.

Free prose never becomes machine authority.

11.10 Structural management principle

Core rule remains:

Red is not invalidation. Green is not an exit. Structure is invalidation.

Before a discretionary exit ask:

If I could not see my P/L, would I still exit this chart right now?

12. Authorization Exceptions and reconciliation

12.1 CRITICAL Authorization Exceptions

Hard violations create durable Authorization Exceptions instead of rewriting broker truth.

Examples:

- exposure above applicable ceiling;
- lifecycle risk-budget violation;

- wrong account;
- wrong direction;
- late opening fill;
- prohibited re-entry;
- management-contract violation;
- unresolved fill/P&L attribution.

A CRITICAL exception:

- blocks further exposure increases;
- remains durable;
- permits risk-reducing operator action at the broker;
- does not automatically reduce/flatten;
- does not create broker-write authority;
- requires explicit reconciliation.

12.2 Late fill after expired authorization

A post-cutoff opening fill on an expired/retired authorization remains broker truth but does not revive the authorization.

Unambiguous late fill:

```
LATE_OPENING_FILL
```

If a plausible later authorization could own the fill:

```
FILL_ATTRIBUTION_UNRESOLVED
```

Exact reassignment is allowed only to an admissible later handoff identified by the system. Assignment then freezes that attribution.

12.3 Reconciliation actions

The live management surface exposes explicit Authorization Exception reconciliation intents such as reviewed broker truth and exact admissible assignment where supported.

These commands reconcile audit/authorization state only. They do not send broker orders.

12.4 General broker coverage/provenance reconciliation

Some downstream coverage/provenance failures can still produce:

```
RECONCILIATION_REQUIRED
LIVE_RECONCILIATION_REQUIRED
```

Ownership remains fail-closed. Do not manually edit persistent state to force release.

The implemented Slice 7 exception-reconciliation workflow should not be misread as a generic automatic resolver for every possible broker coverage/provenance discontinuity.

13. Persistence and recovery

13.1 Server-side V2.4 files

Default local files include:

```
.executionos-v24-state.json  
.executionos-v24-execution-board-handoffs.json  
.executionos-v24-execution-board-handoff-deliveries.json
```

Additional PRETRADE repositories may persist permission/review/OCO/ARM operation and quantity-safety evidence according to the service configuration.

Keep local runtime state private and Git-ignored.

13.2 Browser canonical Execution Board store

Canonical browser store key:

```
execution-v23-store
```

Despite the historical name, it contains both legacy V2.3 and V2.4 downstream namespaces.

13.3 Recovery principles

Ordinary reload/HMR/remount/tab takeover must recover from durable authority rather than visual state.

After leadership acquisition the router rereads:

- latest canonical store;
- broker state;
- available handoff transport.

After LISTENING is durable:

- its boundary is immutable;
- transport loss does not release ownership;
- Schwab evidence loss freezes broker-sensitive conclusions;
- restart does not extend authorization deadlines.

13.4 Never bypass persistence safeguards

Do not manually edit:

- localStorage;
- handoff JSON;
- delivery JSON;
- PRETRADE journals;

- review/quantity-safety evidence;
- Authorization Exception state

to force a lifecycle result.

14. End-of-Day reporting

The EOD reporter combines:

1. Schwab broker execution history;
2. ExecutionOS browser History export.

Schwab is authoritative for fills and position changes.

ExecutionOS History supplies setup/process context.

14.1 V2.4 stop semantics

Current EOD planned-risk and entry-VWAP stop-risk enrichment is origin-aware:

```
V24_HANDOFF      -> v24.effectiveStop
LEGACY_MANUAL_V23 -> originalPlan.structuralStop
```

For V2.4, structural invalidation remains separate provenance and is not substituted for `v24.effectiveStop`.

Slice 7 live managed-stop changes are management state; they do **not** currently rewrite the EOD planned-risk stop basis or replace `v24.effectiveStop` in the EOD enrichment calculation.

The separate PRETRADE 2-ATR quantity-safety policy also does not replace the EOD planned-risk stop basis.

14.2 Enriched EOD procedure

1. Confirm trades are in ExecutionOS History.
2. Keep the same Vite browser origin/profile available.
3. Open:

```
http://localhost:5173/eod-export.html
```

1. Download the ExecutionOS EOD History export.
2. Run:

```
npm run schwab:eod -- --date=YYYY-MM-DD
```

Preferred explicit export path:

```
npm run schwab:eod -- --date=YYYY-MM-DD --executionos=~/.Downloads/executionos-eod-history-YYYY-MM-DD.json
```

Default report:

```
reports/eod/YYYY-MM-DD.html
```

14.3 Verify enrichment

Check:

- intended History export loaded;
- owned vs broker-only counts are plausible;
- setup/process fields appear where expected;
- planned risk/R appears only when supported;
- V2.4 stop basis is correct;
- unmatched cycles remain broker-only.

Private exports/reports should not be committed.

15. Futures / NinjaTrader status

MES/MNQ and supported futures can be represented by Phase 4 sizing and the PRETRADE 2-ATR quantity-safety policy when authoritative instrument metadata is present.

For futures quantity safety, the 2-ATR price distance is converted to ticks with protective upward rounding and multiplied by tick value before deriving the valid quantity maximum.

However:

- live NinjaTrader broker binding is not connected;
- Schwab must not be treated as the futures execution source;
- futures sizing support is not equivalent to futures broker integration;
- a NinjaTrader futures fill will not automatically become LIVE today.

16. Security

Never commit or expose:

- Schwab Client Secret;
- OAuth access/refresh tokens;
- authorization codes;
- unmasked account numbers;
- private broker exports;
- ExecutionOS private History exports;
- V2.4 local runtime state files.

Important local files include:

```
.env.local  
.schwab-tokens.json  
.executionos-v24-state.json  
.executionos-v24-execution-board-handoffs.json  
.executionos-v24-execution-board-handoff-deliveries.json
```

Keep secrets out of browser-exposed `VITE_` variables.

17. Troubleshooting

17.1 Broker offline

```
npm run schwab:monitor  
curl http://127.0.0.1:8787/health
```

`WAITING_FOR_SCHWAB` does not release durable ownership.

17.2 PRETRADE service offline

```
npm run v24:pretrade  
curl http://127.0.0.1:8788/health
```

New PRETRADE/transport work waits. Durable post-LISTENING broker ownership work continues when Schwab evidence remains healthy.

17.3 Schwab authentication fails

```
npm run schwab:auth
```

Verify callback URL, `.env.local`, and token state.

17.4 Router not RUNNING

Interpret health state:

- `WAITING_FOR_SCHWAB` — broker evidence unavailable;
- `WAITING_FOR_PRETRADE` — PRETRADE transport unavailable;
- `WAITING_FOR_ROUTER_LOCK` — another tab owns leadership;
- `PAUSED` — emergency pause active;
- `BLOCKED` — known safety/configuration/capability blocker;
- `STALE` — active leader heartbeat exceeded tolerance;
- `ERROR` — operational failure; durable ownership remains authoritative.

17.5 WAITING candidate does not progress

Check:

- candidate validity window;
- activation/relevance state;
- required manual trigger confirmation;
- trigger evidence type/timeframe;
- structural prerequisites;
- PRETRADE service health;
- current state revision / stale operation conflict.

WAITING candidates are not automatically ARM-authorized.

17.6 Permission will not reach READY/CAUTION

Check:

- trigger satisfaction provenance;
- structural validity;
- if `STRUCTURE = VALID`, structural evidence/reference is non-empty;
- current DSS status;
- Phase 4 status;
- exact account;
- quote/account freshness;
- macro/setup context assessment;
- retryable/integrity blocker reason.

If the UI shows the structural evidence requirement, provide the appropriate evidence/reference; do not bypass backend provenance validation.

17.7 Quantity safety blocks review or selection

Check:

- DSS evaluation contains authoritative ATR evidence;
- ATR is current/valid under the accepted DSS contract;
- Phase 4 evaluation identity matches the reviewed evidence;
- instrument quantity increment/minimum is valid;
- the 2-ATR volatility maximum is nonzero/affordable;
- the selected quantity does not exceed `Final Allowed`.

Do not alter the effective stop to defeat the quantity-safety ceiling.

17.8 ARM returns REVIEW_REQUIRED

The material review package changed during fresh final revalidation.

Review the new package, select quantity again if required, acknowledge CAUTION again if required, and issue a new explicit ARM operation.

Do not force the old package through.

17.9 ARM final allowed quantity decreased

This is valid when fresh Phase 4 or 2-ATR evidence is more restrictive.

The prior reviewed ceiling is non-expanding: fresh evidence can reduce authorization but cannot silently expand it. A higher quantity ceiling requires a new explicit operator review.

17.10 Existing handoff does not progress

Check:

- router health;
- Schwab availability;
- delivery state;
- exact account/symbol;
- browser leadership;
- broker coverage catch-up;
- PREPARED/LISTENING state;
- retirement state.

17.11 Fill did not become owned

Check:

- durable LISTENING;
- first-entry authorization deadline;
- authoritative `executionTime`;
- exact account;
- opening direction/effect;
- continuous coverage;
- ownership journal;
- order identity;
- retirement cutoff.

17.12 Add/re-add is blocked

Check:

- build window / Complete Position Build;
- current live ceiling;
- management contract;
- CRITICAL Authorization Exception;
- current effective stop;
- lifecycle loss budget;
- P/L attribution confidence.

Do not widen/tighten stops merely to manufacture capacity.

17.13 CRITICAL Authorization Exception

Do not delete it or manually alter broker/history state.

Review the broker truth and use only the explicit supported reconciliation intent appropriate to the exception.

Further exposure increases remain blocked while the exception is unresolved.

17.14 General reconciliation required

For `RECONCILIATION_REQUIRED` / `LIVE_RECONCILIATION_REQUIRED` , preserve ownership and investigate the missing coverage/provenance evidence.

Do not edit persistence manually to release the symbol.

17.15 Phase 4 says no affordable size

Do not tighten the stop. Reduce quantity if a smaller valid size exists; otherwise PASS.

17.16 EOD broker-only rows

Check:

1. trade reached History;
2. export was downloaded after completion;
3. same browser origin/profile;
4. correct export loaded;
5. symbol/direction/timing plausibility.

18. Current limitations and deferred work

Current deferred areas include:

- automated Production SOD: deferred / not production-accepted; manual ChatGPT SOD is the current production-generation workflow;
- broker order placement/replacement/cancellation/modification/flattening;
- general broker-write Governor enforcement;
- buying-power/margin eligibility gate;
- aggregate portfolio-heat controls;
- live NinjaTrader futures binding;
- broader asset/currency support;
- cloud persistence/multi-device authority;
- generic automatic resolution for every broker coverage/provenance reconciliation condition;
- AI in the latency-sensitive execution path;
- V3 Management Governor.

Do not bypass these gaps with guessed fallbacks or manual state edits.

19. Repository discipline

Treat:

- `v2.3.0` as the frozen downstream reference;
- `main` as the accepted operating branch for the merged V2.4 implementation;
- `885ee94110e6a91352796c340e6ca40d6d775aff` as the last runtime/code checkpoint and `c23faf5c08f3ba60339197be3687fcad37db0e4b` as the September 15 final integrated main tip;
- the v0.5 baseline + traceability audit as frozen V2.4 design authority for Decisions 22–97;
- the approved PRETRADE Quantity Safety Addendum v0.1 as the September 8 quantity-safety policy authority;
- the September 6 handoff-integration closeout as Slices 1–7 acceptance evidence;
- the September 8 final merge closeout as historical handoff/TODO/regression evidence;
- the September 15 production-core manual-SOD final merge closeout as current integration evidence.

The retired `v24-execution-board-handoff` branch is historical only and should not be recreated for normal operation.

Do not:

- move/delete the `v2.3.0` tag;
- rewrite approved design documents to simulate later implementation state;
- begin V3 without explicit authorization;
- commit credentials/tokens/private trading data;
- commit runtime state files.

A demonstrated implementation defect may be fixed without reopening design when the fix preserves frozen architecture.

A material architecture change requires a new approved future design decision.

20. Recommended daily operating procedure

Before market / before first trade

1. Use `main` and pull it fast-forward-only from `origin/main`.
2. Start `npm run schwab:monitor`.
3. Confirm healthy read-only broker state.
4. Start `npm run v24:pretrade`.
5. Start `npm run dev`.
6. Confirm router health and exact account.
7. Confirm broker positions match reality.

Candidate / PRETRADE

1. Perform the READ and generate the manual ChatGPT SOD with one finalized candidate per individual top-level `MANUAL_AUTHORIZED` JSON bundle.

2. Preserve structure → invalidation → effective stop → risk budget → Phase 4 size → quantity-safety ceiling → explicit selected size.
3. In SOD, load the individual JSON by file, paste, or drop; inspect Preview; explicitly Import into PRETRADE. Confirm ACCEPTED / WAITING , or open the existing candidate after DUPLICATE .
4. Confirm the candidate is within its validity window.
5. Activate/observe trigger evidence as appropriate.
6. If selecting STRUCTURE = VALID , provide a non-empty structural evidence/reference.
7. Let permission run from authoritative evidence.
8. If READY/CAUTION, inspect the current review package.
9. Compare Phase 4 Stop-Risk Max, 2-ATR Volatility Max, Reviewed Ceiling, Final Allowed, and binding constraint.
10. Select an explicit quantity no greater than Final Allowed.
11. If CAUTION, acknowledge the exact package.
12. Verify the exact account and entry mode shown/carried by the current review state.
13. Complete the Section 9.5 pre-ARM checklist.
14. ARM explicitly; the ARM control is the final quantity/direction confirmation.
15. If fresh ARM revalidation reduces Final Allowed or returns REVIEW_REQUIRED, review the new authoritative package rather than forcing the stale one.

After ARM / before fill

1. Confirm Authorized/Execution projection.
2. Let the router process PENDING → CLAIMED → PREPARED → LISTENING.
3. Verify exact account and trade specification.
4. Place the actual equity order only in thinkorswim/Schwab.
5. Remember the finite first-entry authorization deadline.
6. DISCARD only when intentionally relinquishing unfilled authorization.

After first fill

1. Let exact-account broker evidence establish LIVE ownership.
2. Watch ARM ceiling, live ceiling, build window and risk budget.
3. Use Complete Position Build when you intentionally relinquish unused capacity.
4. Use explicit OS effective-stop management only for legitimate structural/risk reasons.
5. Do not assume OS stop changes affect the broker.
6. Record targets/notes as audit state; broker exits remain manual.
7. Treat CRITICAL exceptions as exposure-increase blockers.

Exit / History

1. Broker FLAT moves the owned trade toward EXIT.
2. Complete explicit operator exit classification.
3. History is the normal ownership-release boundary.
4. Export History for enriched EOD reporting as needed.

Outage / restart

Do not interpret service loss as ownership release.

Durable authorization, LISTENING boundaries, retirement cutoffs, LIVE state, exceptions and History remain authoritative across restart according to their contracts.

21. Command reference

Setup / UI

```
git checkout main
git pull --ff-only origin main
npm install
npm run dev
npm run build
npm run preview
npm run v24:pretrade
```

Schwab

```
npm run schwab:auth
npm run schwab:status
npm run schwab:account
npm run schwab:monitor
npm run schwab:history
npm run schwab:replay
npm run schwab:slippage
npm run schwab:state-test
npm run schwab:token-test
npm run schwab:price-history-test
```

EOD

```
npm run schwab:eod
npm run schwab:eod -- --date=YYYY-MM-DD
npm run schwab:eod -- --date=YYYY-MM-DD --executionos=~/.Downloads/executionos-eod-history-YYYY-MM-DD.json
```

V2.4 market data / DSS / risk

```
npm run v24:market-data-test
npm run v24:market-data-probe
npm run v24:dss-live-probe
npm run v24:dss-test
npm run v24:risk-sizing-test
```

Handoff / runtime validation

```
npm run v24:handoff-test
npm run v24:handoff-api-test
npm run v24:broker-provenance-test
npm run v24:handoff-admission-test
npm run v24:v23-compat-test
npm run v24:v23-install-test
npm run v24:fill-ownership-test
npm run v24:retirement-test
npm run v24:activation-test
npm run v24:live-lifecycle-test
npm run v24:store-authority-test
npm run v24:runtime-router-test
npm run v24:router-hardening-test
npm run v24:full-lifecycle-e2e-test
npm run v24:router-browser-test
```

Slice 7 / full integration validation

Focused Slice 7:

```
node --test \
  tests/execution-v24-live-management.test.mjs \
  tests/execution-v24-slice7.test.mjs \
  tests/execution-v24-slice7-final.test.mjs \
  tests/execution-v24-retired-assignment.test.mjs \
  tests/execution-v24-legacy-management-compat.test.mjs
```

Canonical PRETRADE → Execution E2E:

```
node --test tests/execution-v24-pretrade-full-e2e.test.mjs
```

Full repository / final regression components:

```
npm run analytics:test
npm run build
```

Historical September 8 handoff acceptance state:

Focused Slice 7:	26 / 26 PASS
Downstream lifecycle E2E:	1 / 1 PASS
Canonical PRETRADE→Execution E2E:	1 / 1 PASS
September 8 comprehensive regression:	GREEN
Production build:	PASS
Final merged implementation SHA:	26ad8f86d2f0b4af96c186b26f250f4bb10a9dec
Broker writes introduced:	NONE

The final aggregate numeric test count is intentionally not restated because the September 8 acceptance was recorded as an all-green multi-command regression rather than one single aggregate-count artifact.

September 15 production-core validation and the two environment-gated Schwab probes are recorded in [the final merge closeout](#). These are prior implementation acceptance results; this guide refreshes changes documentation only.

22. Documentation map and glossary

22.1 Documentation map

Need	Source
First-time operator / quick start	USER-GUIDE.md — Quick Start
Operate current accepted system on main	USER-GUIDE.md
Current repository overview	README.md
Documentation authority/status	docs/ExecutionOS_Documentation_Index.md
Current vs historical map	DOCUMENTATION-STATUS.md
Frozen V2.4 design authority	docs/ExecutionOS_V2.4_Design_Baseline_v0.5_APPROVED.md
Decision 22-97 traceability	docs/ ExecutionOS_V2.4_Design_Baseline_v0.5_Traceability_Audit_APPROVED. md
PRETRADE quantity-safety policy	docs/ ExecutionOS_V2.4_PRETRADE_Quantity_Safety_Addendum_v0.1_APPROVED.m d
Slices 1-7 accepted closeout	docs/ ExecutionOS_V2.4_Execution_Board_Handoff_Integration_Closeout_2026 -09-06.md
Final merge / TODO / repository closeout	docs/ ExecutionOS_V2.4_Execution_Board_Handoff_Final_Merge_Closeout_2026 -09-08.md
September 15 production-core final merge	docs/ ExecutionOS_V2.4_Production_Core_Manual_SOD_Final_Merge_Closeout_2 026-09-15.md
Manual SOD individual import files	docs/sod/ExecutionOS_V2.4_Manual_SOD_Deliverable_Spec_v1.0.md
Phase 3 closeout	docs/ExecutionOS_V2.4_Phase3_DSS_Closeout_2026-08-31.md
Phase 4 closeout	docs/ExecutionOS_V2.4_Phase4_Risk_Sizing_Closeout_2026-09-01.md
EOD semantics	docs/ExecutionOS_EOD_Report.md

Historical approved documents remain valid approval-time evidence but do not override current validated runtime behavior, the frozen v0.5 authority, the approved quantity-safety addendum, or later accepted closeout records.

22.2 Glossary

Structural invalidation

The structure/price condition that proves the trade thesis wrong.

Structural evidence/reference

Operator-supplied provenance supporting a `STRUCTURE = VALID` assessment. It should identify the observed structure and where/when it was observed. It is required before permission submission in that state; backend provenance enforcement remains authoritative.

Effective stop

The volatility-protected execution stop derived by Phase 3. In LIVE management it may change only through explicit/frozen management authority. The EOD planned-risk basis remains the frozen `v24.effectiveStop` currently exported for the trade; live managed-stop changes do not rewrite that planned-risk basis.

DSS evaluation

Immutable Phase 3 evaluation identified by `dssEvaluationId`. Its authoritative ATR evidence is also used by the separate PRETRADE quantity-safety policy.

Risk evaluation

Immutable Phase 4 evaluation identified by `riskEvaluationId`.

Phase 4 Stop-Risk Max

Largest valid pre-entry quantity fitting actual expected-entry-to-effective-stop risk under the 0.5% account-equity budget.

2-ATR Volatility Max

Separate PRETRADE quantity maximum derived from a 2-ATR adverse-move stress exposure using the same max-dollar-risk budget.

Reviewed Ceiling

Candidate/version quantity ceiling frozen at explicit review. Fresh ARM revalidation may reduce it but may not increase it without a new explicit review.

Final Allowed

Current maximum quantity allowed by the minimum of fresh Phase 4 max, fresh 2-ATR volatility max, and the applicable reviewed ceiling. This is the operator's maximum selectable quantity; selecting less is allowed when valid for the instrument.

Review package

Material authorization-critical snapshot shown before ARM. Material changes produce a new identity.

ARM ceiling

Immutable `selectedQuantity` maximum simultaneous authorized exposure frozen at ARM.

Live ceiling

Current maximum exposure permitted after build-window expiry/completion. It may ratchet downward but not above the ARM ceiling.

V2.4 ARMED authorization

Immutable authorization freeze containing exact candidate, DSS, risk, account, quantity and management/entry provenance. It does not place an order.

Handoff

Immutable transfer contract from V2.4 authorization into downstream execution ownership.

PREPARED

Durable local pre-fill reservation before authoritative listening begins.

LISTENING

Durable pre-fill reservation with immutable `executionListeningAt`.

First-entry authorization deadline

Immutable time boundary by which a qualifying first fill must have authoritative broker `executionTime`.

Complete Position Build

Explicit operator action that relinquishes never-used build capacity and freezes the lower live ceiling.

Authorization Exception

Durable record that broker truth violated or cannot be confidently attributed within authorization constraints. CRITICAL exceptions block exposure increases.

LATE_OPENING_FILL

Opening fill at/after an expired authorization cutoff; broker truth is retained without reviving the expired authorization.

LIVE_RECONCILIATION_REQUIRED

Fail-closed downstream ownership state when broker continuity/provenance becomes unprovable.

Execution state

Structural management classification such as `VALID`, `THREATENED`, or `INVALID`.

Governor

Planned future V3 management-policy layer. V3 has not started.

Living-document maintenance rule

Update this guide whenever accepted branch/release state, startup, candidate/permission/ARM workflow, risk or quantity-safety semantics, handoff/ownership, Slice 7 management, persistence, broker safety boundary, supported instruments, EOD procedure, CLI surface, or first-time operator workflow changes.

Do not let this guide drift away from validated application behavior.